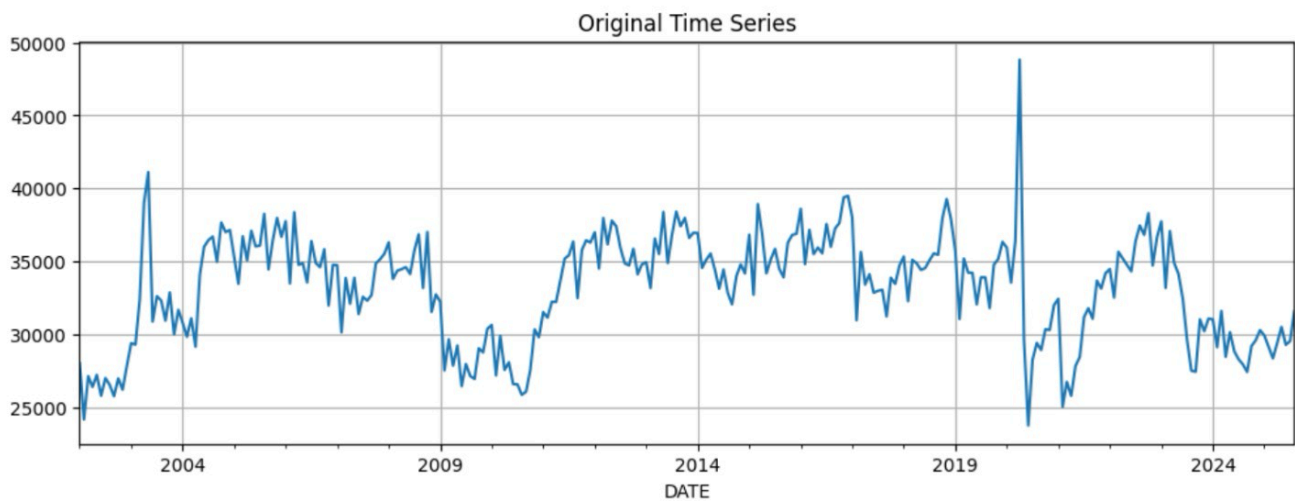
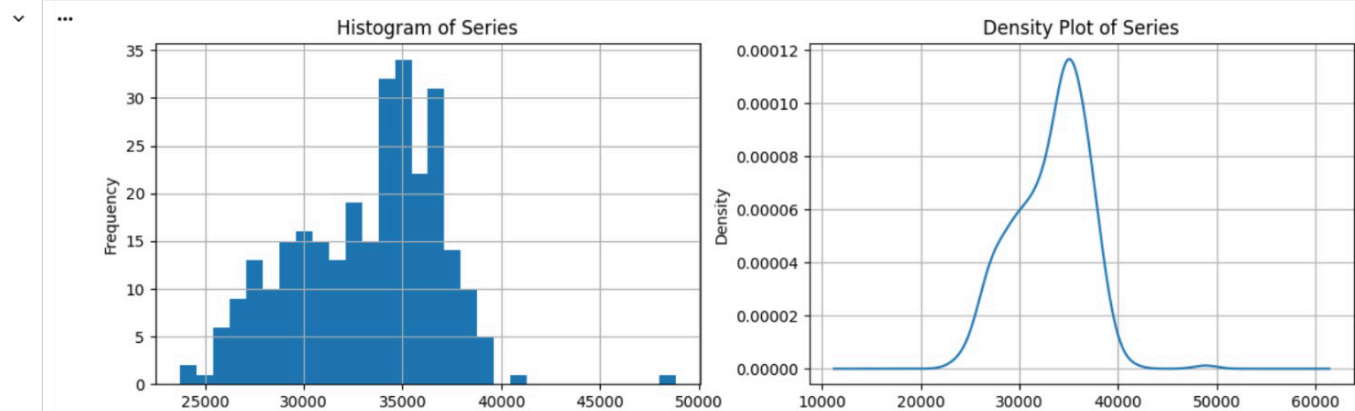


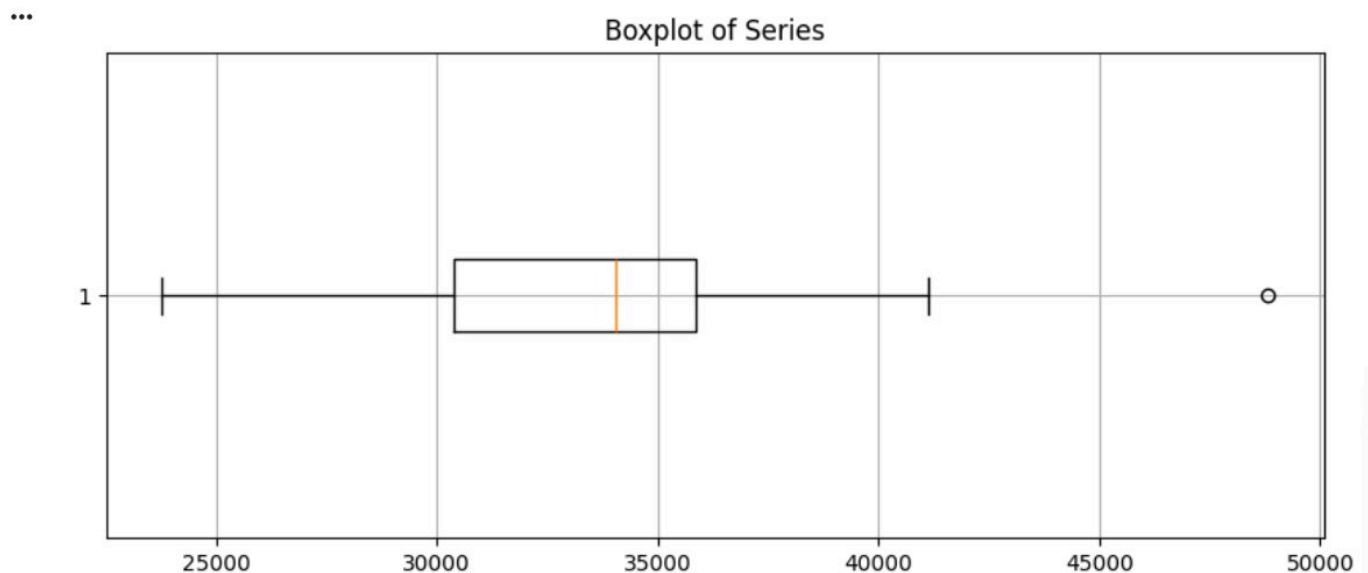
```
series.plot(title="Original Time Series")
plt.show()
```



```
[ ] fig, ax = plt.subplots(1, 2, figsize=(14, 4))
series.plot(kind='hist', bins=30, ax=ax[0], title='Histogram of Series')
series.plot(kind='kde', ax=ax[1], title='Density Plot of Series')
plt.show()
```



```
[ ] plt.figure(figsize=(10, 4))
plt.boxplot(series.dropna(), vert=False)
plt.title("Boxplot of Series")
plt.show()
```



✓ إحصاءات وصفية

هذه الإحصاءات تساعدنا على فهم القيم المركزية والانتشار.

[]



```
display(series.describe())
```



...

Value

count	284.000000
--------------	------------

mean	33197.013810
-------------	--------------

std	3662.936429
------------	-------------

min	23757.512200
------------	--------------

25%	30377.515975
------------	--------------

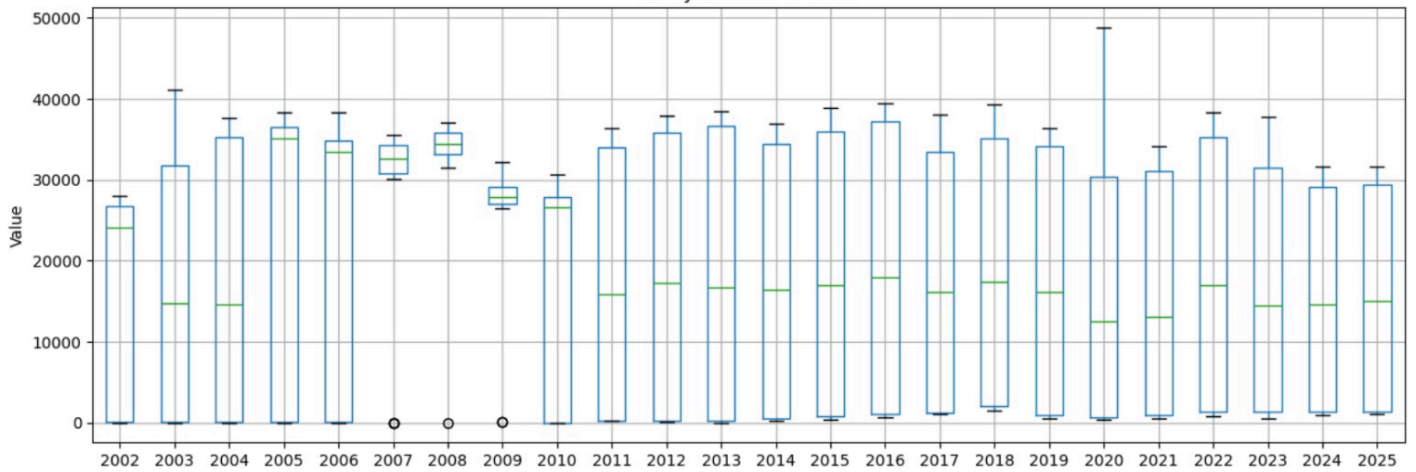
50%	34044.405250
------------	--------------

75%	35863.064000
------------	--------------

max	48826.671800
------------	--------------

dtype: float64

Yearly Box Plot of Value



▶ `adf_report(series, "ADF on Original Series")`

```

... ===== ADF on Original Series =====
ADF Statistic : -3.703424
p-value       : 0.004065
Used Lag      : 12
N Observations: 271
Critical Values:
    1%: -3.454713
    5%: -2.872265
   10%: -2.572485
Conclusion    : السلسلة ثابتة ✓
  
```

Moving Average

```

oil_df_feature["Moving Average 3"] = oil_df_feature["Value"].shift(1).rolling(window=3).mean()
oil_df_feature["Moving Average 6"] = oil_df_feature["Value"].shift(1).rolling(window=6).mean()
oil_df_feature["Moving Average 12"] = oil_df_feature["Value"].shift(1).rolling(window=12).mean()
  
```

```
oil_df_feature.head(10)
```

DATE	Energy Product Name	Flow breakdown	Unit of Measure	Value	Year	Month	Day	Day of Week	Quarter	Day Name	Lag 1	Lag 3	Lag 7	Rolling Mean 7	Rolling Std 7	Expanded
2002-01-01	Crude Oil	Exports	KL	28024.1025	2002	1	1	1	1	Tuesday	NaN	NaN	NaN	NaN	NaN	
2002-02-01	Crude Oil	Exports	KL	24172.4697	2002	2	1	4	1	Friday	28024.1025	NaN	NaN	NaN	NaN	28024.102
2002-03-01	Crude Oil	Exports	KL	27141.8805	2002	3	1	4	1	Friday	24172.4697	NaN	NaN	NaN	NaN	26098.286

```
oil_df_feature.dropna(inplace=True)
oil_df_feature
```

DATE	Energy Product Name	Flow breakdown	Unit of Measure	Value	Year	Month	Day	Day of Week	Quarter	Day Name	Lag 1	Lag 3	Lag 7	Rolling Mean 7	Rolling Std 7	Expanded
2003-01-01	Crude Oil	Exports	KL	29394.2574	2003	1	1	2	1	Wednesday	27846.6724	26984.1648	25798.9125	26594.077471	748.261149	26593.943
2003-02-01	Crude Oil	Exports	KL	29305.2243	2003	2	1	5	1	Saturday	29394.2574	26213.8701	27008.8079	27107.698171	1205.642651	26809.351
2003-03-01	Crude Oil	Exports	KL	32415.4981	2003	3	1	5	1	Saturday	29305.2243	27846.6724	26540.5895	27435.757657	1459.875898	26987.628
2003-04-01	Crude Oil	Exports	KL	39025.0882	2003	4	1	1	2	Tuesday	32415.4981	29394.2574	25765.5251	28275.030314	2304.100133	27349.486
2003-05-01	Crude Oil	Exports	KL	41124.3601	2003	5	1	3	2	Thursday	39025.0882	29305.2243	26984.1648	30169.253614	4397.022085	28079.211
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2025-04-01	Crude Oil	Exports	KL	29409.5202	2025	4	1	1	2	Tuesday	28359.2483	29931.4764	27430.1250	29137.060929	974.746674	33253.115
2025-05-01	Crude Oil	Exports	KL	30513.0529	2025	5	1	3	2	Thursday	29409.5202	29144.9649	29202.0414	29419.831671	619.364622	33239.388

```
plt.ylabel('Value')
plt.title('Time Series with Moving Averages')
plt.show()
```



Power Transform

1- Log

```
log_oil_df = np.log(oil_df['Value'])  
log_oil_df
```

Value

DATE

2002-01-01	10.240820
------------	-----------

2002-02-01	10.092970
------------	-----------

2002-03-01	10.208833
------------	-----------

2002-04-01	10.181296
------------	-----------

2002-05-01	10.211915
------------	-----------

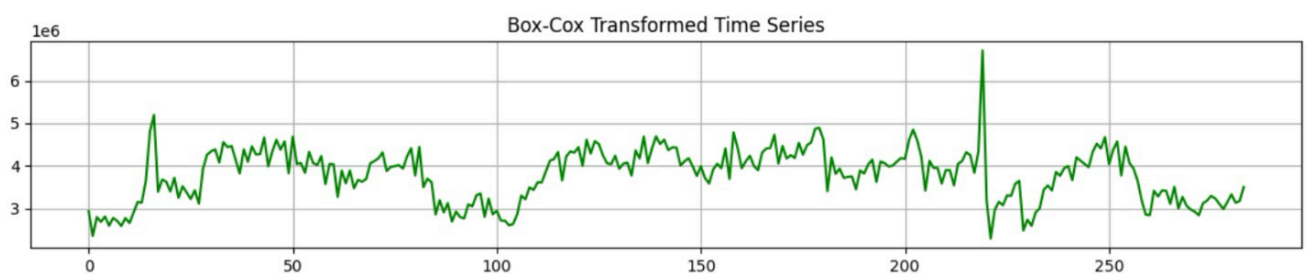
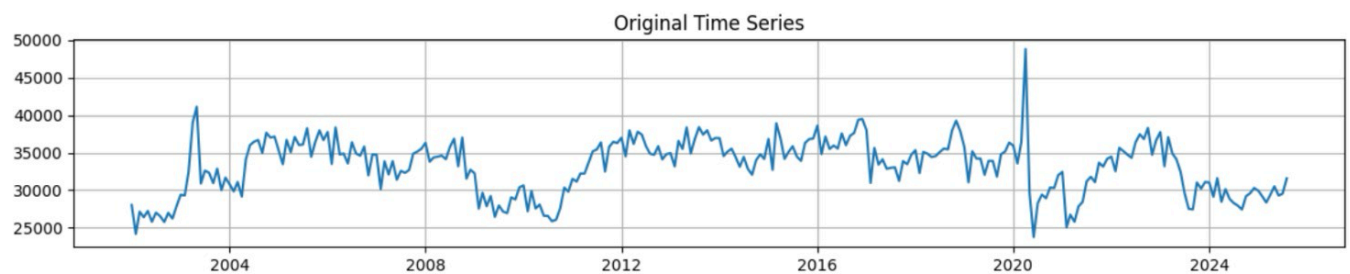
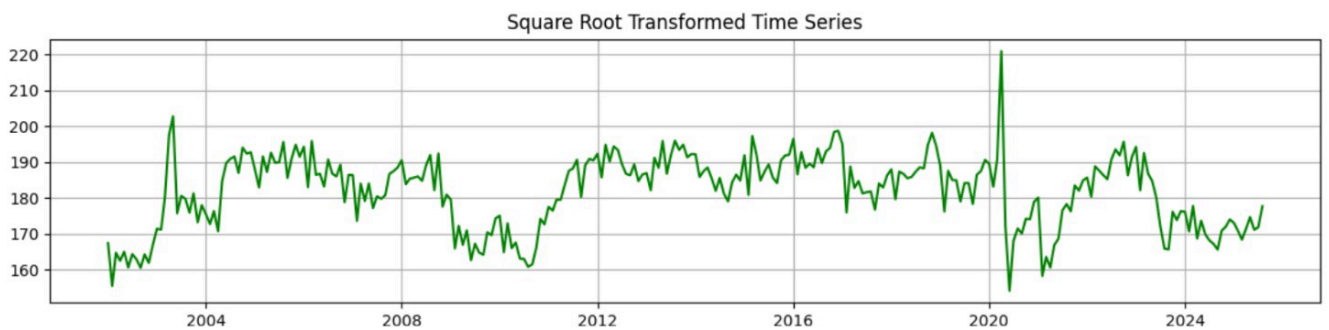
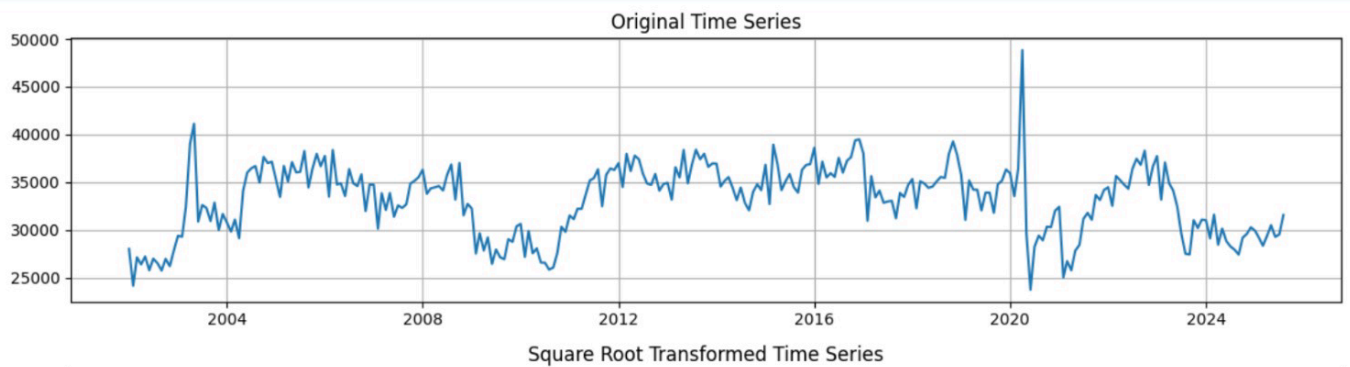
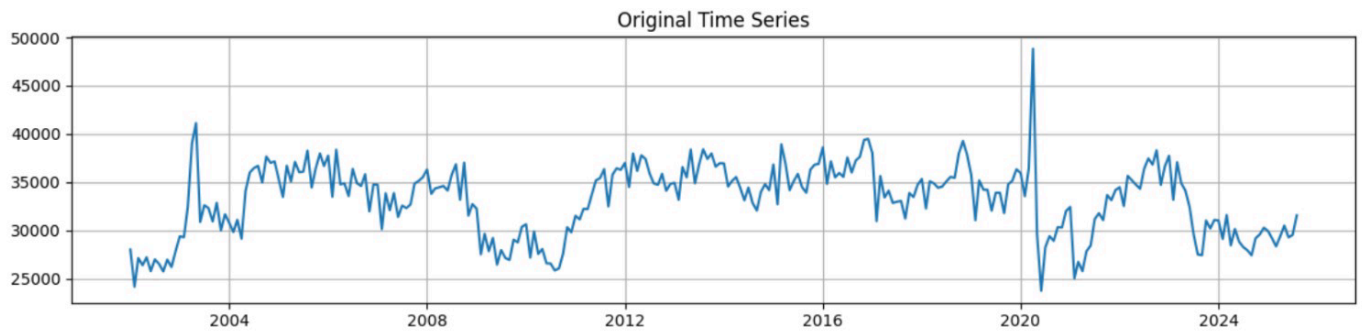
...

...

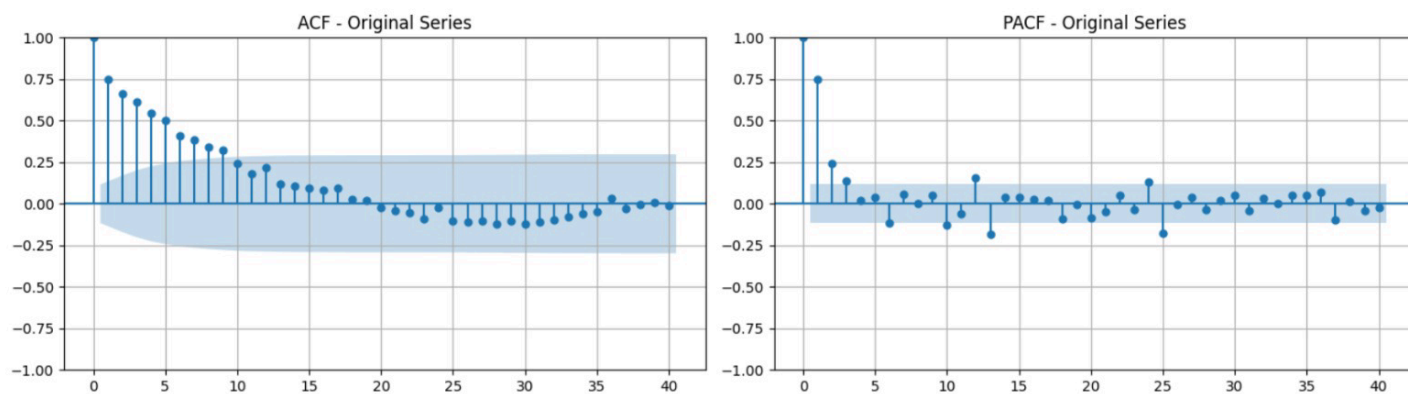
2025-04-01	10.289074
------------	-----------

2025-05-01	10.325910
------------	-----------

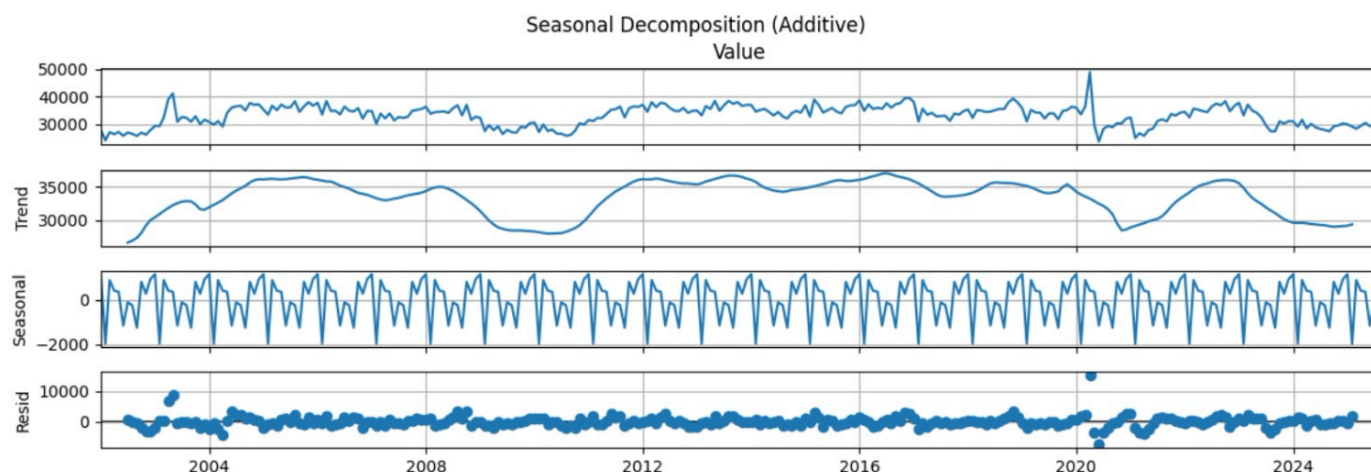
2025-06-01	10.335011
------------	-----------



...



...



تقسيم البيانات

سنحتفظ بأخر 30% من البيانات للاختبار حتى نقيس قدرة النموذج على التنبؤ في بيانات لم يرها أثناء التدريب.

[]

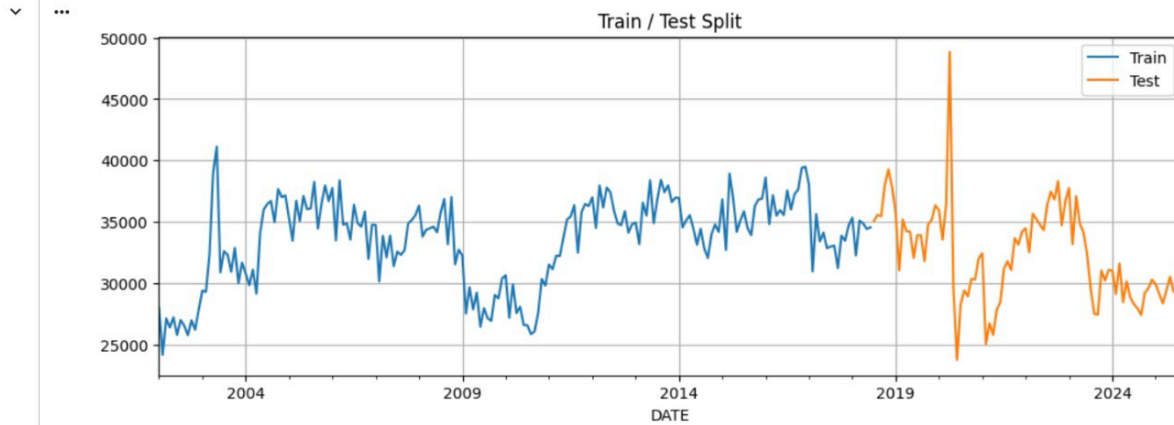
```
split_idx = int(len(series) * 0.7)
train = series.iloc[:split_idx]
test = series.iloc[split_idx:]

print("Train size:", len(train))
print("Test size :", len(test))
```

▼

Train size: 198
Test size : 86

```
[ ] ▶ plt.figure(figsize=(12, 4))
train.plot(label="Train")
test.plot(label="Test")
plt.title("Train / Test Split")
plt.legend()
plt.show()
```



... 2004 2009 2014 2019 2024

Machine Learning استيراد مكتبات

Regression Problem. سنستخدم مجموعة من نماذج الانحدار المناسبة للتنبؤ بالقيم المستمرة، لأن مسألة التنبؤ الزمني هنا هي

```
[ ] from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
```


[]

```

def create_ml_features(series, n_lags=12):
    df_ml = pd.DataFrame({"Value": series.copy()})

    # Lag Features
    for lag in range(1, n_lags + 1):
        df_ml[f"lag_{lag}"] = df_ml["Value"].shift(lag)

    # Rolling Features
    df_ml["rolling_mean_3"] = df_ml["Value"].shift(1).rolling(window=3).mean()
    df_ml["rolling_std_3"] = df_ml["Value"].shift(1).rolling(window=3).std()

    df_ml["rolling_mean_6"] = df_ml["Value"].shift(1).rolling(window=6).mean()
    df_ml["rolling_std_6"] = df_ml["Value"].shift(1).rolling(window=6).std()

    # Time Features
    df_ml["month"] = df_ml.index.month
    df_ml["quarter"] = df_ml.index.quarter
    df_ml["year"] = df_ml.index.year

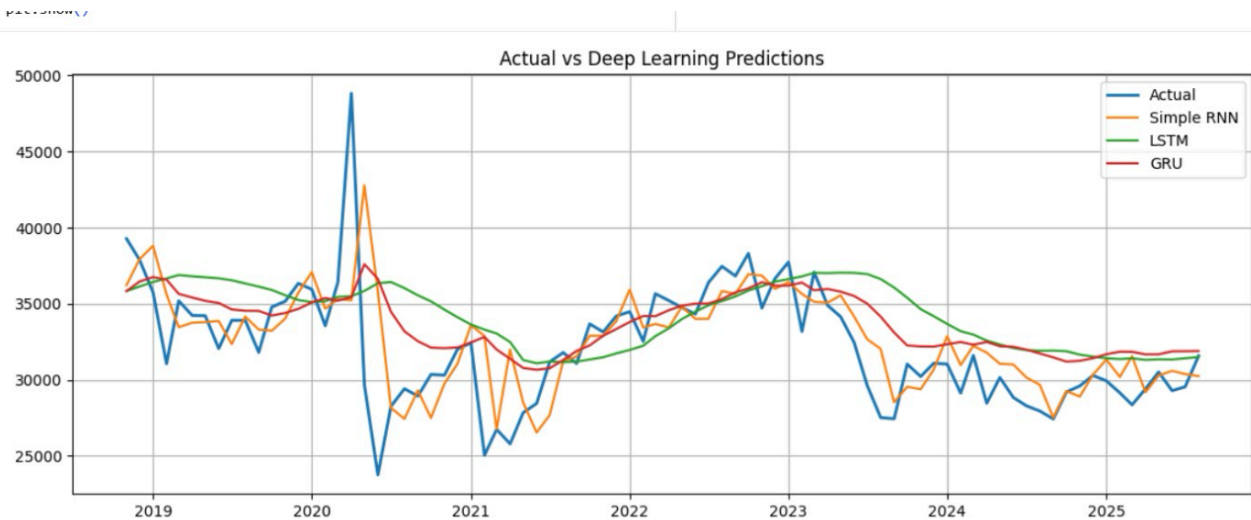
    # rolling والـ shifts حذف القيم المفقودة الناتجة عن الـ
    df_ml = df_ml.dropna()

    return df_ml

ml_data = create_ml_features(series, n_lags=12)

print("Machine Learning dataset shape:", ml_data.shape)
display(ml_data.head())

```



...

	Model	RMSE	MAE
4	Gradient Boosting	2686.975735	1706.062496
5	XGBoost	2700.059232	1640.385268
3	Random Forest	2711.020883	1761.169268
1	Linear Regression	3275.489265	1971.074970
0	Baseline_ML	3306.645185	1993.994913
2	Decision Tree	3486.130084	2496.432012

...

	Model	RMSE	MAE	AIC	Description
0	Baseline	3241.014615	1943.214438	NaN	Persistence baseline
2	ARIMA	4381.495579	3461.173592	3542.411763	Handles non-seasonal trend
3	SARIMA	4439.538181	3497.903454	2740.106183	Handles trend + seasonality
1	ARMA	5467.295392	4368.963912	3531.415663	ARMA on differenced series



Next steps:

[Generate code with results](#)

[New interactive sheet](#)



saudi-oil-project.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

Share M

RAM Disk



- RMSE
- MAE

وكلما انخفضت القيم كان الأداء أفضل.

```
[32] ✓ 0s
d1_results = pd.DataFrame({
    "Model": ["Simple RNN", "LSTM", "GRU"],
    "RMSE": [rnn_rmse, lstm_rmse, gru_rmse],
    "MAE": [rnn_mae, lstm_mae, gru_mae]
})

d1_results = d1_results.sort_values("RMSE", ascending=True)
display(d1_results)
```

	Model	RMSE	MAE
0	Simple RNN	3159.404299	1895.088832
2	GRU	3476.927353	2477.247880
1	LSTM	4095.398191	3131.120123

Next steps:

[Generate code with d1_results](#)

[New interactive sheet](#)

